

6.3 - Métricas que enxergam tendências

Counter, gauge, histogram e o conjunto mínimo

Estava no ar, mas estava fora

- **LunaLog monitorava apenas uptime do serviço**
 - No ar ou fora do ar, nada além disso

LUNALOG

A LunaLog acompanhava uma única coisa: o serviço estava no ar ou não. O primeiro incidente com o cliente enterprise destruiu essa ilusão de controle. O serviço respondia, sim, mas com latência de oito segundos para 30% das requisições, o que para o usuário era idêntico a estar fora. Ana resumiu na reunião seguinte: medir disponibilidade sem latência, taxa de erro e throughput é como avaliar a saúde de um paciente olhando só para o pulso. Faltavam as métricas que contam a história real da experiência.

Os quatro tipos fundamentais de métrica

Cada tipo responde uma pergunta diferente

- **Counter só cresce e mede acúmulos**
 - Total de requisições, erros ou bytes enviados
- **Gauge sobe e desce, mede estado atual**
 - Uso de memória, conexões abertas, tamanho de fila
- **Histogram distribui observações em faixas de valor**
 - Base para calcular percentis de latência como o P95
- **Summary calcula percentis no próprio cliente**
 - menos flexível que histogram para agregar entre instâncias

RED e USE: o conjunto mínimo

RED (foco no usuário)

- Rate: requisições por segundo
- Errors: taxa de falhas das requisições
- Duration: latência por percentil
- Ideal para serviços que atendem requisições

USE (foco no recurso)

- Utilization: quanto do recurso está em uso
- Saturation: fila e trabalho represado
- Errors: erros do próprio recurso
- Ideal para CPU, memória, disco e rede

Cardinalidade e nomeação importam

- Nomeie a métrica com a unidade no final
- Padrão Prometheus: `http_request_duration_seconds`

Cada combinação de labels cria uma nova série temporal. Um label de alta cardinalidade, como ID de usuário, pode multiplicar suas séries por milhares e estourar o sistema de métricas. Escolha labels com poucos valores possíveis.

- Use labels para dimensões finitas: método, rota, status



Com logs e métricas, você sabe que uma requisição chegou lenta ao usuário. Mas quando ela atravessa oito serviços antes de responder, onde exatamente o tempo foi perdido? A métrica diz que está lento, ela não diz onde. Para isso existe um terceiro pilar, capaz de reconstruir o caminho inteiro de uma requisição como uma única história.

Tracing distribuído: spans, traces e correlação